

Instruction for Training a Model for Identification

1. Environment Setup

To run the project, the following must be installed:

- Python 3.10 or higher
- Ultralytics YOLOv8 library
- Roboflow library

`pip install LIBRARY_NAME`

2. Data Preparation

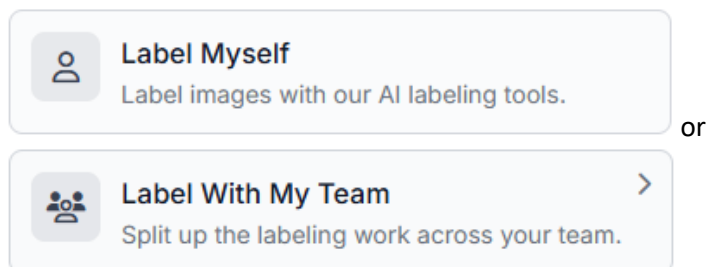
1. Create a project in Roboflow



2. Upload images of colloidal clusters



3. Annotate (label) objects in the images



4. Create dataset splits (train / validation / test)



5. Export the dataset in YOLOv8 format



After exporting, you will receive a code snippet that allows automatic dataset download on the server where the training is performed.



×

What are you planning to do with this dataset?

☐ Share with others

☐ Analyze or experiment

☒ Train a model

☐ Other

Continue





×

Train your model directly

No need to download. Train your model in a few clicks and unlock more train, deploy, and analytics potential.

With a Roboflow model, you can:

 Instance Segmentation Model
my_model

 detection_visualization

 count_objects

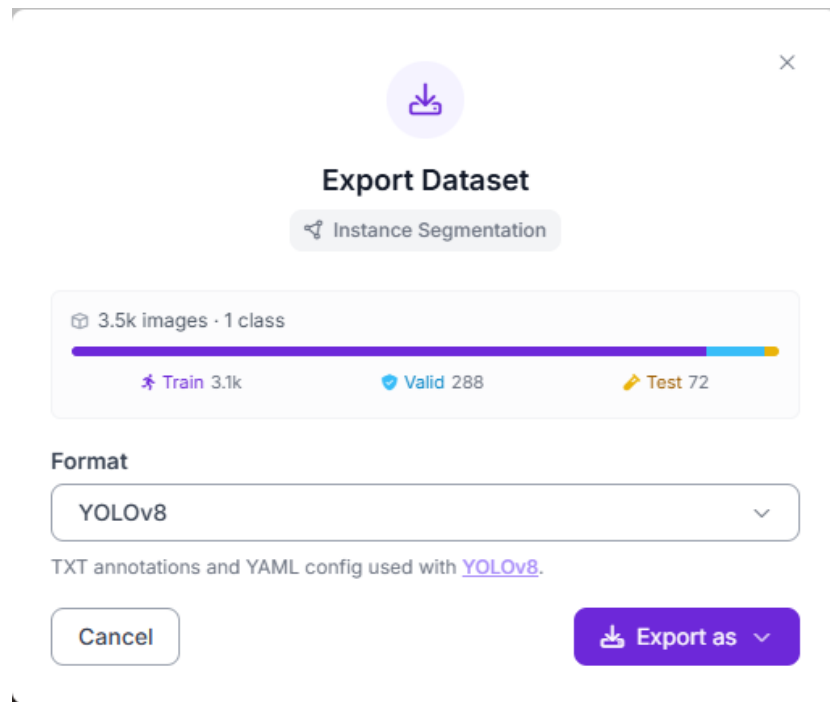
Build intelligent multimodal vision systems

← ● ● ● →

||

Download Dataset

Train a Model



* export as code snippet

Example of the generated snippet:

```
!pip install roboflow

from roboflow import Roboflow

rf = Roboflow(api_key="YOUR_ROBOFLOW_API_KEY")

project = rf.workspace("YOUR_ROBOFLOW_WORKSPACE").project("YOUR_PROJECT_NAME")

dataset = project.version(1).download("yolov8")
```

After export, the following structure will be created:

```
dataset/
├── train/
├── valid/
├── test/
└── data.yaml
```

3. Script Configuration

In the training script, the following parameters must be specified:

Roboflow API key (already used during dataset download)

```
rf = Roboflow(api_key="YOUR_ROBOFLOW_API_KEY")
```

Workspace name

```
rf.workspace("YOUR_ROBOFLOW_WORKSPACE")
```

Project name

```
.project("YOUR_PROJECT_NAME")
```

Path to the dataset

```
data="PATH_TO_YOUR_DATA.YAML_FILE"
```

Example:

```
data="dataset/data.yaml"
```

Project version

```
VERSION = 1
```

4. Model Training

Training is started using:

```
model.train
```

During training, the following information is displayed:

- epoch number
- loss values
- evaluation metrics

5. Training Results

After training is completed, results are automatically saved in:

```
runs/
```

```
└─ segment/
```

```
    └─ train/
```

Main files include:

- best.pt — best performing model
- last.pt — model from the last epoch
- results.png — training curves
- confusion_matrix.png — confusion matrix

6. Проверка качества модели

To test the trained model, use the following code:

```
from ultralytics import YOLO
```

```
model = YOLO("best.pt")
```

```
results = model.predict(
```

```
    source="PATH_TO_YOUR_IMAGE.jpg",
```

```
    save=True
```

```
)
```

After execution, an output image will be generated with detected objects visualized

7. Uploading the Model to Roboflow

After successful training, the model can be deployed to Roboflow:

```
project.version(VERSION).deploy(  
    model_type="yolov8",  
    model_path="PATH_TO_TRAIN_RESULTS"  
)
```

where

```
model_path="runs/segment/train"
```

After upload, the model becomes available in the Roboflow interface for further use

You can see uploaded version in

A screenshot of a button in the Roboflow interface. The button is light purple with a darker purple border and contains a small icon of a document with a checkmark followed by the text "Versions".

Versions